

**UNIVERSIDADE DO VALE DO ITAJAÍ
ESCOLA POLITÉCNICA
CURSO DE ENGENHARIA DE COMPUTAÇÃO
MICROCONTROLADORES**

**SISTEMA DE VIGILÂNCIA E RASTREAMENTO
AUTOMÁTICO**

por

Henry José

Itajaí (SC), junho de 2026

**UNIVERSIDADE DO VALE DO ITAJAÍ
ESCOLA POLITÉCNICA
CURSO DE ENGENHARIA DE COMPUTAÇÃO
MICROCONTROLADORES**

**SISTEMA DE VIGILÂNCIA E RASTREAMENTO
AUTOMÁTICO**

por

Henry José

Relatório apresentado como requisito parcial da disciplina de Microcontroladores do Curso de Engenharia de Computação para análise e aprovação.
Professor Responsável: Paulo Roberto Valim

Itajaí (SC), junho de 2026

SUMÁRIO

1 INTRODUÇÃO.....	4
1.1 DEFINIÇÃO DO PROBLEMA.....	4
1.2 SOLUÇÕES EXISTENTES.....	5
1.3 SOLUÇÃO PROPOSTA	6
1.4 ANÁLISE DE VIABILIDADE	7
2 PROJETO	8
2.1 VISÃO GERAL.....	8
2.2 PREMISSAS.....	<i>Erro! Indicador não definido.</i>
2.3 ANÁLISE DE REQUISITOS.....	9
2.3.1 Requisitos funcionais.....	9
2.3.2 Requisitos não funcionais.....	10
2.4 ARQUITETURA DE HARDWARE	11
2.5 ARQUITETURA DE SOFTWARE	12
2.6 PLANEJAMENTO	13
2.7 CRONOGRAMA	<i>Erro! Indicador não definido.</i>
2.8 ANÁLISE DE RISCOS.....	15
3 DESENVOLVIMENTO	16
3.1 IMPLEMENTAÇÃO	16
3.2 VERIFICAÇÃO.....	19
3.3 RESULTADOS.....	19
4 CONSIDERAÇÕES FINAIS / CONCLUSÕES	24

1 INTRODUÇÃO

1.1 DEFINIÇÃO DO PROBLEMA

Durante os eventos da Segunda Guerra Mundial, a Inglaterra precisava de contramedidas a possibilidade de invasão por parte da Alemanha nazista, que tinha acabado de tomar o território da França. Apesar de possuírem uma defesa natural, o oceano, que tornavam invasões algo complexo e arriscado, ainda existia o risco de invasões aéreas ao território inglês.

Logo, se fez necessário, com urgência, a criação de uma forma de detecção a aproximação de aeronaves alemãs. Nesse contexto, foi desenvolvida uma tecnologia que utilizava antenas que transmitiam ondas em alta frequência. A reflexão das ondas, causada pela passagem de objetos metálicos, como aviões, provocava ruídos na frequência portadora, indicando aos Aliados que algo se aproximava.

Essas informações tinham valor crucial no contexto bélico, pois permitia que pilotos de combate da RAF (Real Força Aérea) fossem despachados para combate. A essa tecnologia se deu o nome Radio Detection and Ranging, ou como é comumente chamada, RADAR, e serviria como uma das principais armas contra a superioridade bélica germânica.

Ainda hoje, com o aumento dos voos comerciais, o RADAR foi empregada ao controle de tráfego aéreo. Com o passar de novas guerras, a tecnologia foi aprimorada, e atualmente está sendo utilizada pelos Estados Unidos, Israel e Irã, ou Ucrânia e Rússia, em seus respectivos conflitos. Em janeiro de 2026, a Ucrânia recebeu um radar modelo Lanza 3D de Indra, capaz de detectar missões e drones russos de longo alcance. Em março do mesmo ano, o Irã destruiu um radar estratégico dos EUA avaliado em US\$ 1,1 bilhão no Qatar, capaz de detectar mísseis a 5.000 KM. Isso demonstra que a tecnologia ainda possui vulnerabilidades e precisa de medidas de proteção.

A detecção antecipada de ameaças bélicas aéreas é crucial para o rumo de guerras travadas entre nações. A necessidade de tecnologia RADAR cada vez mais sofisticadas capazes de lidar com armas modernas vem crescendo à medida que a guerra avança. Países de primeiro mundo investem bilhões em tecnologia de radar e em medidas para defender as mesmas, como mísseis teleguiados que vão em direção a possíveis ameaças contra o radar. Desse modo, sistemas de vigilância e rastreamento automático, ou radar antimíssil, se fazem cada vez mais necessário.

1.2 SOLUÇÕES EXISTENTES

1.2.1 Sea-Based X-Band Radar (SBX)

O SBX é um sistema de defesa de mísseis balísticos americano projetado para detectar e estabelecer informações precisas de rastreamento em mísseis balísticos, discriminar ogivas de mísseis de chamarizes e detritos, fornecer dados para atualizar interceptores terrestres em voo e avaliar os resultados de tentativas de interceptação. O custo do equipamento foi avaliado em US\$ 900 milhões.

O radar na SBX está alojado sob o radome grande e branco e é considerado o maior e mais sofisticado radar de banda X (2 – 4 GHz) dirigido eletromecanicamente do mundo, com aproximadamente 45.000 módulos de transmissão/recepção formando o feixe de radar. O feixe de radar é capaz de detectar um objeto do tamanho de uma bola de beisebol a até 2.500 milhas de distância. O radar também usa 69.632 circuitos multi-seccionais para transmitir, receber e amplificar sinais, e a elevação em que o radar está posicionado no exterior a plataforma permite rastrear objetos enquanto eles voam em direção, sobre e longe da embarcação.

1.2.2 Sistema de Combate Aegis

O sistema digital americano de comando e controle desenvolvido para coordenar radares, sensores e armamentos em tempo real, com frequência Banda S (3 – 3,5 GHz). Esse sistema permite que o navio detecte ameaças aéreas a grandes distâncias, rastreie simultaneamente centenas de alvos, calcule automaticamente soluções de interceptação e coordene o lançamento de mísseis defensivos. O custo foi avaliado entre US\$ 400 a US\$ 600 milhões por embarcação com essa tecnologia.

O radar principal utilizado é o AN/SPY-1D, um radar multifuncional de alta potência que opera em banda S (2 – 4 GHz) e pode monitorar grandes áreas do espaço aéreo e marítimo ao redor da embarcação. Essa capacidade de vigilância transforma o destróier em um verdadeiro centro de comando de defesa aérea para grupos navais.

1.2.3 Iron Dome

A Cúpula de Ferro (Iron Dome, em inglês) é um renomado sistema móvel de defesa aérea desenvolvido e operado por Israel, projetado especificamente para proteger o território contra-ataques aéreos de curto alcance. Em funcionamento desde 2011, a principal função do

sistema é rastrear, identificar e interceptar projéteis de artilharia, morteiros e foguetes disparados de distâncias que variam entre 4 e 70 quilômetros. Sua tecnologia atua em quaisquer condições climáticas e utiliza uma lógica inteligente de triagem: o sistema calcula rapidamente a trajetória do alvo no ar e só dispara um míssil interceptador se prever que o projétil inimigo cairá em uma zona habitada ou em infraestruturas críticas, evitando desperdícios em áreas desertas.

O funcionamento do sistema é segmentado em três pilares fundamentais: detecção, controle e interceptação. O coração tecnológico da Cúpula de Ferro é o radar multimissão ELM-2084, fabricado pela ELTA Systems (subsidiária da Israel Aerospace Industries), que opera na banda S do espectro eletromagnético — utilizando frequências que variam entre 2 e 4 GHz. Esse radar de alta precisão varre o espaço aéreo constantemente para captar lançamentos. Uma vez detectado o projétil, as informações são processadas pelo centro de gerenciamento de combate e controle de armas, que aciona as unidades de disparo baseadas em solo, compostas por baterias que carregam até 20 mísseis interceptadores conhecidos como Tamir.

Em termos financeiros e de capacidade operacional, o custo estimado de implantação de cada bateria completa gira em torno de 50 milhões de dólares, enquanto o custo unitário de cada míssil interceptador Tamir disparado fica entre 40 mil e 150 mil dólares. Quanto à sua capacidade balística, embora tenha sido concebido primordialmente contra foguetes não guiados e ameaças de baixa altitude e curto alcance, o sistema tem demonstrado evolução tecnológica substancial ao longo dos anos.

1.3 SOLUÇÃO PROPOSTA

O sistema de vigilância e rastreamento possui alcance de 4 metros, frequência de 40 kHz, angulação de até 180°, e fornece os dados de distância e angulação do objeto detectado. Ao detectar objetos com o radar, aciona um laser que aponta para o centro do alvo. O protótipo utiliza dois servomotores, um para movimentar o sensor ultrassônico para detectar objetos e o outro para disparo do laser de detecção. A movimentação é limitada ao próprio eixo dos servomotores.

A solução atua em sistemas de pequeno porte e curto alcance. Pode ser utilizada em residências para detecção e localização de intrusos, além de alertar e registrar aproximações.

Quadro 1 – Comparativo da solução proposta com as soluções existentes

Nome	Alcance	Frequência	Mobilidade	Angulação	Papel	Custo Estimado
SBX	2.000 km	2 - 4 GHz	Fixo	360° (raio de 2.000 km).	Detectar, rastrear e discriminar ameaças, enviar dados.	US\$ 900 milhões.
Sistema de Combate Aegis	190 km	3 – 3,5 GHz	Móvel no mar.	360° (raio de 190 km).	Detecte, calcular soluções de interceptação, coordene o lançamento de mísseis defensivos	US\$ 400 - 600 milhões.
Iron Dome	474 km	2 - 4 GHz	Móvel	360° (raio de 190 km).	Proteger a população e as cidades de Israel contra ataques aéreos de curto alcance.	US\$ 50 milhões.
Sistema de vigilância	4 m	40 KHz.	Fixo	0 – 180°	Vigilância e rastreamento automático	US\$ 25

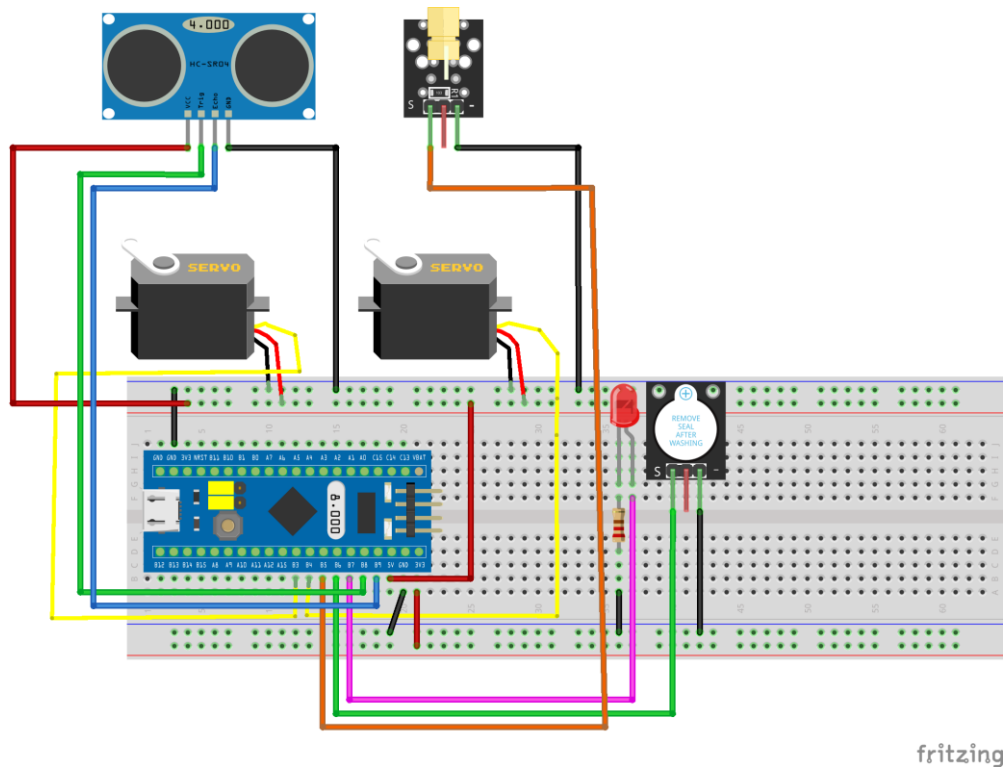
1.4 ANÁLISE DE VIABILIDADE

A implementação pode ser dividida em quatro etapas: compra dos componentes, montagem do protótipo, programação no STM32, testes e ajustes. Sobre a primeira etapa, os componentes são baratos e o custo é para o prazo de entrega é de uma semana até a chegada dos componentes. Com a compra antecipada, durante a etapa de apresentação, a chegada dos componentes é sincronizada a segunda semana, que é a de implementação, logo não haverá impactos significativos no tempo.

A etapa mais crítica pode ocorrer durante a programação da lógica do radar, que envolve a detecção de objetos e a implementação de uma interface gráfica para a sua visualização, distância e angulação, bem como a programação do laser para localização, pois como envolve programação utilizando cálculos de angulação, requer um código mais sofisticado. Essa etapa pode se estender até os testes e ajustes, todavia não ocupará tempo sobressalente.

2 PROJETO

2.1 VISÃO GERAL



O sistema será composto por dois servomotores SG90, um sensor ultrassônico HC-SR04, um módulo laser KY-008, um buzzer ativo KY-012 e um LED indicador, todos controlados pelo microcontrolador STM32F103C8T6.

O primeiro servomotor será responsável pela movimentação horizontal do sensor ultrassônico, realizando uma varredura contínua em um campo de visão de 180°. Durante essa varredura, o sensor HC-SR04 efetuará medições periódicas de distância para identificar a presença de objetos no ambiente monitorado.

O segundo servomotor permanecerá em posição de espera enquanto não houver detecção de objetos. Quando um objeto for identificado pelo sensor ultrassônico, o sistema determinará a posição angular correspondente à detecção e acionará o segundo servomotor para posicionar o módulo laser na mesma direção do alvo detectado.

Simultaneamente, o sistema ativará o buzzer como alerta sonoro e fará o LED piscar de forma intermitente, indicando visualmente a presença do objeto. Esses alertas permanecerão ativos enquanto o objeto continuar sendo detectado dentro da área monitorada. Após a remoção do objeto, o sistema retornará ao modo de varredura normal.

Além do sistema embarcado, será desenvolvida uma interface gráfica para computador responsável por exibir, em tempo real, os dados coletados pelo sensor. A interface apresentará um gráfico de sonar representando a área monitorada, a distância medida até o objeto detectado, o ângulo atual de varredura do sensor e registros de eventos do sistema.

Também serão disponibilizados dois logs distintos: um para registrar os objetos detectados durante a operação e outro para registrar os objetos removidos da área monitorada, permitindo o acompanhamento histórico das ocorrências observadas pelo sistema.

2.2 ANÁLISE DE REQUISITOS

2.2.1 Requisitos funcionais

- RF01: o sistema deve detectar e mapear obstáculos num campo de visão de 180° e alcance de até 4 m;
- RF02 o sistema deve direcionar um apontador laser para a posição angular do obstáculo detectado;
- RF03: o sistema deve acionar um alarme sonoro sempre que um obstáculo for detectado, mantendo-o ativo enquanto o obstáculo permanecer na área monitorada;
- RF04: o sistema deve acionar uma luz de sinalização intermitente sempre que um obstáculo for detectado, mantendo-a ativa e piscando enquanto o obstáculo permanecer na área monitorada;
- RF05: o sistema deve conter uma interface gráfica, exibida no computador, com os seguintes elementos visuais:
 - a. Representação gráfica do sonar, exibindo em tempo real os obstáculos detectados no campo de visão de 180°.
 - b. Registro (log) contendo data/hora, identificador do objeto, distância e ângulo da detecção, bem como data/hora, identificador do objeto, distância e ângulo da interceptação pelo laser.

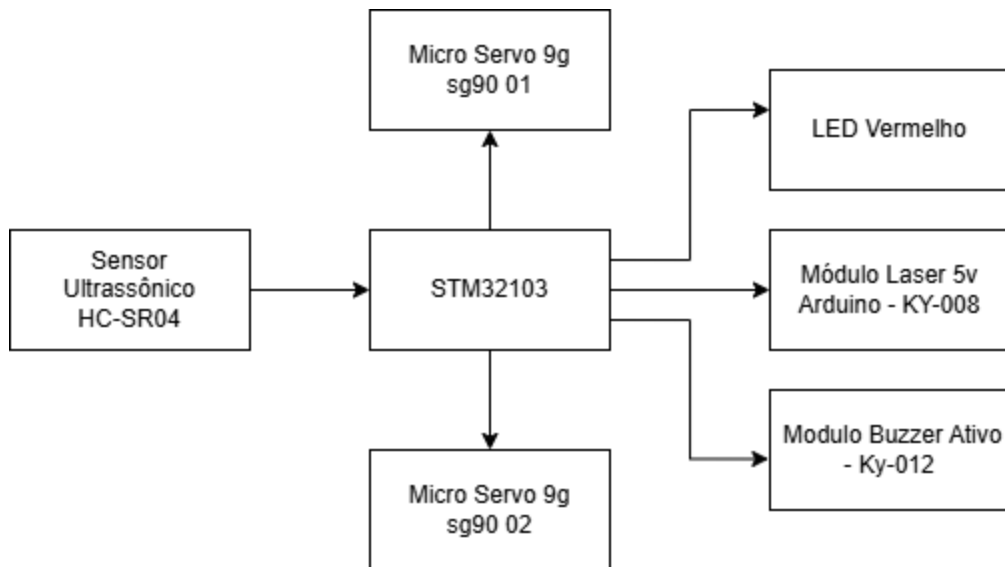
- c. Exibição em tempo real do ângulo e da distância correspondentes ao obstáculo atualmente detectado.
- RF06: o sistema deve realizar varredura contínua do ambiente em um setor angular de 180° utilizando um servo motor.

2.2.2 Requisitos não funcionais

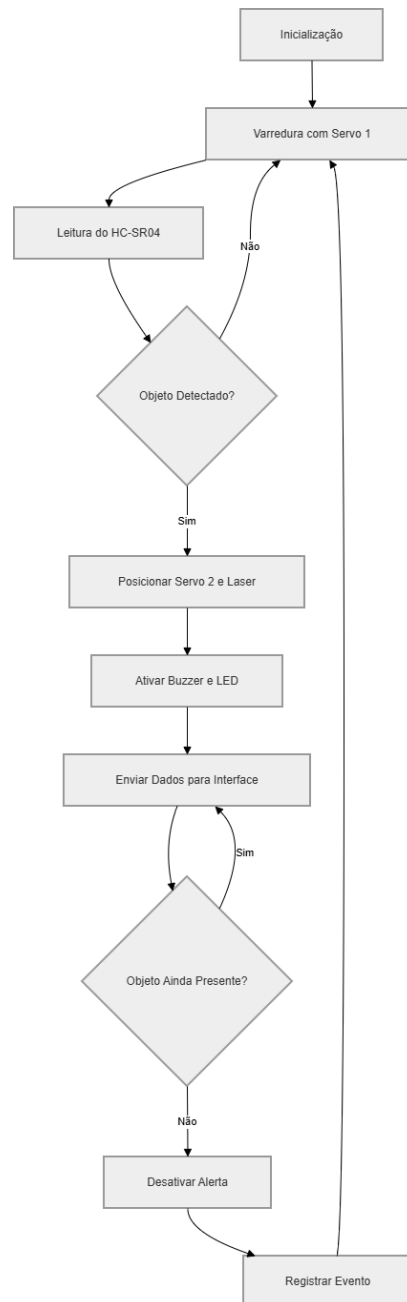
- RNF01: o sistema será prototipado no microcontrolador STM32F103C8;
- RNF02: o código da aplicação será escrito em linguagem C;
- RNF03: será utilizado o ambiente de desenvolvimento STM32CubeIDE Version: 1.17.0;
- RNF04: o código da interface gráfica deve ser feito em linguagem python ou HTML5;
- RNF05: será utilizado o ambiente de desenvolvimento Visual Studio Code como auxiliar na implementação da interface.
- RNF06: O protótipo poderá utilizar estruturas mecânicas e componentes auxiliares para fixação e posicionamento dos dispositivos eletrônicos.
- RNF07: O sistema deverá operar com alimentação de 5 V para os dispositivos periféricos e 3,3 V para o microcontrolador.
- RNF08: O firmware deverá utilizar comunicação serial para troca de informações entre o microcontrolador e a interface gráfica.
- RNF09: O sistema deverá apresentar tempo de resposta inferior a 500 ms entre a detecção de um obstáculo e a atualização da interface gráfica.
- RNF10: O código-fonte deverá ser modularizado, permitindo manutenção e futuras expansões do sistema.
- RNF11: O sistema deverá ser capaz de operar continuamente por pelo menos 30 minutos sem falhas de funcionamento durante os testes.
- RNF12: O protótipo deverá ser montado utilizando componentes de baixo custo e fácil aquisição no mercado nacional.

- RNF13: A interface gráfica deverá exibir de forma clara as informações coletadas pelo sensor ultrassônico e o posicionamento angular do servo motor.
- RNF14: O código deverá seguir as boas práticas de programação embarcada, incluindo uso de comentários, nomenclatura padronizada e separação em módulos funcionais.

2.3 ARQUITETURA DE HARDWARE



2.4 ARQUITETURA DE SOFTWARE



2.5 LISTA DE MATERIAIS

Quadro 2 – Lista de materiais

Item	Qtd	Disponível	Responsável
Microcontrolador STM32F103C8 BluePill	01	Sim	Henry José
Protoboard 830 Pontos	01	Sim	Henry José
Micro Servo Motor 9g SG90	02	Sim	Henry José
Módulo Laser 5v Arduino - KY-008	01	Sim	Henry José
Ky-012 Modulo Buzzer Ativo	01	Sim	Henry José
Sensor Ultrassônico HC-SR04	01	Sim	Henry José
Suporte Para Sensor Ultrassônico HC-SR04 Cor Vermelha	01	Sim	Henry José
Jumpers	60+	Sim	Henry José

2.6 PLANEJAMENTO

Quadro 3 – Plano de verificação

Requisito	Procedimento de verificação/teste	Resultado esperado
RF01	Verificar se servo motor está girando num campo de 180°; verificar se alcance do sensor ultrassônico está de acordo com as especificações.	Servo gira em 180°; sensor está detectando obstáculos no alcance correto.
RF02	Verificar se a posição angular do laser é satisfatória para rastrear o obstáculo, a posição do laser está apontando para o obstáculo de forma satisfatório ou é necessário o cálculo do centro do objeto detectado.	Servo aponta para o centro do objeto ou para a borda.
RF03	Posicionar obstáculos na frente do sensor ultrassônico para verificar se o buzzer aciona de forma quase instantânea.	Buzzer emite som de aviso.
RF04	Posicionar obstáculos na frente do sensor ultrassônico para verificar se o led pisca enquanto o objeto está posicionado.	Led piscando.
RF05	A representação gráfica do sonar deve condizer com a posição real do servo; o registro do log deve conter a hora correta, o a distância angular condizente com a realidade e distância correta. Isso vale para o laser. Erro tolerando de 5%.	Exibição das informações do obstáculo no log.
RF06	Verificar se está usando o servo motor do modelo correto e se as varreduras estão no campo de 180°.	Servo motor atuando no ângulo de 180°
RNF01	Verificar se a placa é a original e não uma cópia	Placa original.
RNF02	Verificar se nas especificações do projeto no STM IDE Cube está marcado como uso da linguagem C ao invés da C++.	Configurada com linguagem C.
RNF03	Verificar a versão do software atende as especificações.	Está na versão 1.17.0
RNF04	Verificar se as linguagens possuem ferramentas de comunicação ou integração com o STM32.	As linguagens possuem essas ferramentas para integrar corretamente linguagens gráficas.
RNF05	Verificar se o ambiente está devidamente configurado para atender o uso das ferramentas na prototipação.	O VSCode possui as extensões para auxiliar com a programação em C, Python ou HTML5.
RNF06	Verificar se os componentes auxiliares se integram corretamente com os componentes eletrônicos.	Os componentes auxiliares encaixam corretamente com os componentes eletrônicos.
RNF07	Utilizar um multímetro para verificar a tensão nos componentes.	Os componentes estão recebendo a alimentação correta.
RNF08	Verificar se os arquivos .ioc está com as portas configuradas corretamente para garantir o funcionamento correto da comunicação serial com a interface gráficas. Verificar como está configurado a API.	O arquivo está com todas as portas devidamente configuradas.
RNF09	Utilizar bibliotecas de verificação de tempo de para identificar o tempo de execução. Verificar se possível seções de código com custo computacional elevado.	O código está devidamente otimizado e os tempos de execução estão pertinentes.
RNF10	Verificar se o código está com a separação entre arquivos de cabeçalho e arquivo de implementação para cada componentes, comentários pertinentes.	Arquivos separados, organizados e com comentários pertinentes.

RNF11	Tentar o protótipo durante um tempo para ver se continua responsivo e atuando corretamente. Verificar se há descarte dos componentes durante o uso contínuo.	O protótipo funciona corretamente por mais de 30 minutos.
RNF12	Verificar custo-benefício, frete dos componentes necessários	A maioria dos componentes é barato, porém o frete pode ser mais salgado.

Quadro 4 – Plano de avaliação

Métrica	Procedimento de a valiação	Resultado esperado
Corrente	25mA STM32	Funcionamento responsivo.
Energia	3.3 V para microcontrolador, 5V para Servo e Sensor ultrassônico.	Funcionamento correto dos componentes.
Tempo de resposta	500 ms de resposta a partir da detecção do obstáculo	Sistema Responsivo.

2.7 ANÁLISE DE RISCOS

Quadro 5 – Análise de riscos

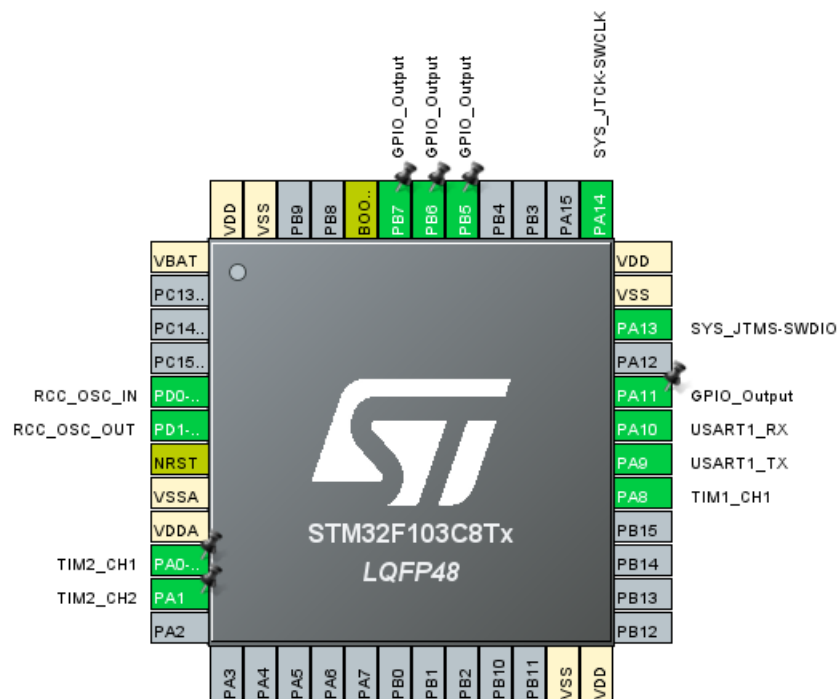
Risco	Probabilidade	Impacto	Gatilho	Plano de contingência
Queimar chip do microcontrolador.	Média	Alto	Alimentar acidentalmente a placa com 5v.	Verificar a possibilidade de pegar uma placa emprestada.
Micro servo motor não operar corretamente.	Baixa	Médio	Uso inadequado do componente. Curto-circuito. Defeito de fábrica.	Pegar outro emprestado ou adaptar o projeto para funcionar com somente um.
Tensão ou corrente insuficiente.	Baixa	Baixo	Alimentar os componentes com fontes que fornecem pouca carga.	Trocar a fonte de alimentação.
Sensores ou atuadores não funcionarem corretamente.	Média	Médio	Uso inadequado do componente. Curto-circuito. Defeito de fábrica.	Verificar possibilidade de troca do componente ou adaptador o projeto.
Memória insuficiente.	Médio	Alto	Gerenciamento de memória inadequado no STM32.	Técnicas de otimização de código.

3 DESENVOLVIMENTO

Este capítulo é exclusivo para o relatório final (implementação) e não deve ser incluído no relatório do projeto (apenas no documento final).

3.1 IMPLEMENTAÇÃO

A configuração da STM adotada nesse projeto utilizou de 1 temporizador para os servos motores (TIM2), 1 temporizador para o sensor ultrassônico (TIM1), USART TX e RX para comunicação serial e 4 portas GPIO Output para laser, buzzer, LED e Trigger do sensor ultrassônico. Os prescaler adotado para os servos foi de 72-1 Mhz e Counter Period de 2000-1 e para o ultrassônico prescaler de 72-1 e Counter Period de 0xffff. O Baud Rate do USAR foi de 115200, sem paridade.



A filosofia de programação adotada foi a orientada a objetos, na qual cada componente ou entidade do projeto recebeu um tipo exclusivo que englobou as variáveis necessárias para a sua implementação. Tome como exemplo o SG90, o servo:


```
typedef struct {
    TIM_HandleTypeDef *htim; //timer, deve ser atrelado ao timer configurado no arquivo .ioc
    uint32_t channel; //canal, deve ser atrelado ao canal configurado no timer do arquivo .ioc
    uint16_t currentPWM; //Atual CCR, usando para fazer o servo girar, deve ser convertida para graus em caso de calculo trigonométrico
    int8_t direction; //direção (1 indo e -1 voltando)
    int8_t quadrant; //quadrante, para corrigir o laser
    uint32_t lastUpdate; //tempo de resposta mínimo, serve para evitar que ande no tempo correto
    uint8_t scanFinished; //Varredura de 180 graus completa
} SG90_t; //Nome do modelo do sensor ultrassônico
```

Após criar todos os componentes e entidades necessário para o projeto, eles foram englobados numa entidade só, chamada prototype, que funciona como a variável global central do projeto:

```
typedef struct{
    SG90_t scanServo; //Servo motor responsável fazer o sensor ultrassônico capturar alvo dentro de um certo intervalo angular
    SG90_t attackServo; //Servo responsável por posicionar o laser em direção ao alvo detectado pelo sensor ultrassônico.
    HCSR04_t scanSonar; //Sensor ultrassônico responsável por detectar alvo numa certa distância
    alert_t attackLaser; //Laser que aponta para o alvo, é acionado quando o servo de attack se posicionar em direção ao alvo
    alert_t alertBuzzer; //Emite um bipe intermitente para alertar que um alvo foi detectado
    alert_t alertLed; //Pisca para alertar que um alvo foi detectado
    target_t target; //Informações do alvo
}sonar_attack_t; //Nome geral do projeto

extern sonar_attack_t prototype; //Variável que é usada em outro arquivo, mas que precisa existir nesse para poder ser chamada no main
```

A utilização de MACROS e diretivas para tornar o código legível. Além de variáveis globais que representam os temporizadores. Destaque para as macros ÂNGULO PARA CCR e CCR PARA ÂNGULO que são responsáveis pelo fluxo de conversão de CCR utilizado pelo servo para girar para ângulo, utilizado na exibição no monitor serial e cálculos trigonométricos. TICK EXPIRED garante comportamento não bloqueante.

```
#include "sonar_de_ataque.h"
#ifdef SRC_SONAR_DE_ATAQUE_C
#define SRC_SONAR_DE_ATAQUE_C

/*Diretivas, Macros e Globais*/
sonar_attack_t prototype;
#define SERVO_MIN_CCR 250 //0°
#define SERVO_MED_CCR 750 //90°
#define SERVO_MAX_CCR 1250 //180°
#define FIRST_QUADRANT 1
#define SECOND_QUADRANT 2
#define BACK 1
#define FORTH -1
#define DISTANCE_BETWEEN_COMPONENTS 10.0f
#define LIMIT_RANGE 10
#define TICK_EXPIRED(lastTick, interval) ((HAL_GetTick() - (lastTick) >= (interval)) ? ((lastTick) = HAL_GetTick(), 1) : 0)

#define ANGULO PARA CCR(ang) \
    (SERVO_MIN_CCR + \
    (((uint32_t)(ang)) * \
    (SERVO_MAX_CCR - SERVO_MIN_CCR) / 180U))

#define CCR PARA ANGULO(ccr) \
    (((uint32_t)(ccr)) - SERVO_MIN_CCR) * \
    180U / \
    (SERVO_MAX_CCR - SERVO_MIN_CCR)

#define DEG_TO_RAD(x) ((x) * 3.14159265359f / 180.0f)
#define RAD_TO_DEG(x) ((x) * 180.0f / 3.14159265359f)

static uint8_t alertState = 0; //alternar o estado do buzzer e do LED
static uint32_t lastAlertUpdate = 0; //verifica a passagem de tempo entre o alternar de um estado e de outro do buzzer e do LED
static uint32_t last_measure = 0; //
static uint32_t last_print = 0;
```

O método utilitário essencial é o **CalculateTargetAngle**, que calcula a o ângulo no qual o servo motor com o laser deve apontar.

```
static float CalculateTargetAngle(void)
{
    float a = prototype.target.Distance;           // distancia medida pelo sonar
    float beta = DEG_TO_RAD((float)prototype.target.Angle); // angulo do sonar em rad
    float c = DISTANCE_BETWEEN_COMPONENTS;        // distancia entre sonar e laser

    // Coordenadas do alvo no referencial do sonar
    float x = a * cosf(beta);
    float y = a * sinf(beta);

    // Corrige para o referencial do laser
    float thetaLaser = atan2f(y, (x + c));

    float ang = RAD_TO_DEG(thetaLaser);

    // Limita entre 0 e 180
    if (ang < 0.0f) ang = 0.0f;
    if (ang > 180.0f) ang = 180.0f;

    return ang;
}
```

Outro método utilitário importante é o **HAL_TIM_IC_CaptureCallback**, que possui o cabeçalho nativo no STM e permite a implementação de captura de eventos elétricos, como borda de subida e descida. Esse método é responsável pela captura de tempo do sensor ultrassônico num intervalo de us.

Fluxo do projeto

- **Sonar_Attack_Init**: inicializa todos os componentes do prototype
- **Sonar_Attack_Scan**: faz o servo do ultrassônico girar em aproximadamente 180° indo e voltando
- **Sonar_Attack_Neutralize**: faz o servo de ataque girar na direção do alvo e acionar o laser.
- **Sonar_Attack_HCSR04_Read**: responsável pelas leituras distância do sensor ultrassônico
- **Sonar_Attack_Target**: captura informações do alvo, como distância e posição angular
- **Sonar_Attack_Monitor**: envia dados para o monitor serial.
- **Sonar_Attack_Loop**: engloba todos os anteriores numa máquina de estados finitos.

3.2 VERIFICAÇÃO

Apresente os procedimentos de verificação utilizados e os resultados obtidos com a aplicação do plano de verificação. Transcreva o quadro apresentado na seção anterior e acrescente uma coluna para descrever os resultados obtidos. É importante analisar o grau de atendimento de cada requisito. Também apresente imagens de experimentos de verificação, tais como diagramas de formas de onda de simulação, capturas de tela de console e equipamentos de instrumentação, fotos da operação do protótipo. Procure descrever como a imagem evidencia o cumprimento dos requisitos.

De início, todos os componentes foram testados separadamente para garantir que estavam se comunicando corretamente com a STM.

3.3 RESULTADOS

Durante o desenvolvimento do sistema de sonar de ataque, foram realizados diversos testes com o objetivo de validar o funcionamento dos módulos de hardware e da lógica de software responsável pela detecção e neutralização de alvos. Os principais testes realizados são descritos a seguir.

1. Teste individual dos periféricos

Inicialmente, todos os componentes foram testados individualmente para verificar seu correto funcionamento:

- **Servo de varredura:** validou-se o movimento entre 0° e 180°, confirmando o correto funcionamento do PWM gerado pelo STM32.
 - **Servo de ataque:** foi testado separadamente, verificando-se que respondia corretamente aos valores de PWM enviados.
 - **Sensor ultrassônico HC-SR04:** confirmou-se a correta medição da distância utilizando o modo Input Capture do temporizador.
 - **Laser, LED e buzzer:** os dispositivos foram acionados individualmente por meio das GPIOs, verificando-se seu funcionamento elétrico.
-

2. Teste da varredura do ambiente

Após a validação individual dos componentes, implementou-se a varredura contínua do ambiente utilizando o servo responsável pelo sensor ultrassônico.

Foram realizados testes para verificar:

- movimentação contínua do servo;
- sincronização entre o movimento do servo e as leituras do HC-SR04;
- conversão entre o valor de PWM (CCR) e o ângulo correspondente.

Os testes demonstraram que o sistema era capaz de mapear corretamente a posição angular das leituras realizadas.

3. Desenvolvimento da máquina de estados (FSM)

Foi implementada uma máquina de estados para controlar o comportamento do sistema.

A primeira versão possuía três estados:

- **TARGET_SEARCHING**
- **TARGET_TRACKING**
- **TARGET_LOCKED**

Durante os testes observou-se que o sistema permanecia indefinidamente no estado **TARGET_TRACKING**, pois a transição para **TARGET_LOCKED** somente ocorria quando o alvo deixava de ser detectado durante a varredura.

Como consequência, o servo de ataque, o laser, o LED e o buzzer nunca eram acionados.

4. Alteração da lógica de detecção

Para solucionar o problema anterior, foi implementada uma nova estratégia baseada na menor distância encontrada durante uma varredura completa.

O algoritmo passou a:

1. iniciar uma nova varredura;
2. armazenar continuamente a menor distância encontrada;
3. registrar o ângulo correspondente;
4. ao término da varredura, selecionar automaticamente o alvo mais próximo.

Essa abordagem permitiu que o sistema passasse corretamente para o estado **TARGET_LOCKED**, acionando os dispositivos de neutralização.

Entretanto, foi observado um novo inconveniente: o sistema somente detectava o alvo após o término completo da varredura, aumentando significativamente o tempo de resposta.

5. Desenvolvimento de uma nova estratégia de aquisição do alvo

Para reduzir a latência do sistema, foi desenvolvida uma lógica baseada na confirmação do alvo durante a própria movimentação do servo.

Nessa estratégia:

- o sistema continua realizando a varredura continuamente;
- ao detectar um objeto dentro da distância limite, inicia um processo de confirmação;
- caso o alvo permaneça presente durante leituras consecutivas, ocorre imediatamente a transição para o estado **TARGET_LOCKED**, sem necessidade de aguardar o término da varredura.

Essa abordagem tornou o sistema significativamente mais responsivo.

6. Testes de acionamento do servo de ataque

Após a implementação do estado **TARGET_LOCKED**, verificou-se que:

- o LED era acionado corretamente;
- o buzzer funcionava normalmente;
- o laser era ligado conforme esperado.

Entretanto, o servo de ataque permanecia imóvel.

Foram realizados diversos testes de depuração, incluindo:

- impressão do ângulo calculado para o servo;
- impressão do valor de CCR enviado ao temporizador;
- verificação da execução da função `Sonar_Attack_Neutralize()`;
- confirmação de que o estado **TARGET_LOCKED** estava sendo alcançado.

Os resultados indicaram que a lógica de software estava sendo executada corretamente.

7. Investigação do cálculo geométrico

Foi implementada uma função para compensar o deslocamento físico existente entre o sensor ultrassônico e o laser.

Os testes mostraram que o cálculo retornava corretamente o ângulo compensado, indicando que a função matemática não era responsável pela ausência de movimento do servo.

8. Hipótese de limitação da alimentação elétrica

Considerando que ambos os servos eram acionados simultaneamente, levantou-se a hipótese de insuficiência de corrente na alimentação.

Foram observados os seguintes indícios:

- o servo de varredura permanecia em movimento contínuo;

- o servo de ataque não apresentava qualquer movimento;
- LED, buzzer e laser continuavam funcionando normalmente.

Como servos apresentam elevado consumo de corrente durante a movimentação, especialmente no instante inicial, concluiu-se que a alimentação poderia estar limitando o funcionamento do segundo servo.

Essa hipótese permaneceu em investigação.

9. Identificação de falsos positivos

Durante os testes práticos verificou-se que, ocasionalmente, o sistema acionava o laser, o LED e o buzzer mesmo sem a presença de obstáculos.

Foram identificadas algumas possíveis causas:

- leituras espúrias do sensor HC-SR04;
- ecos múltiplos provenientes do ambiente;
- reutilização da última distância válida quando uma nova medição não era concluída;
- ausência de filtragem das leituras antes da mudança de estado;
- atualização assíncrona da variável de distância pela interrupção do temporizador.

Essas observações motivaram a proposta de implementação de mecanismos adicionais de filtragem e validação das medições.

4 CONSIDERAÇÕES FINAIS / CONCLUSÕES

4.1 CONSIDERAÇÕES FINAIS

O STM32 é uma plataforma robusta para prototipagem e implementação de projetos sofisticados. A plataforma permite a integração de diversos componentes, a utilização de timer, comunicação serial, PWM e cálculos trigonométricos, o que garante uma ferramenta completa para o desenvolvimento de projetos que atendam os mais diversos requisitos.

Algumas limitações observadas foram principalmente na questão de alimentação dos componentes que causou o mal funcionamento durante os testes por falta de alimentação correta, pois todos estavam usando alimentação diretamente o ST-Link, dos pinos de 5V e 3.3V. Logo, é recomendado o uso de uma fonte externa que garante tensão e corrente adequadas para todos os componentes.

Além disso, é importante garantir fornecedor confiável de componentes eletrônicos, pois pode ocorrer de alguns componentes estarem com mal funcionamento. Nesse projeto, alguns dos componentes apresentaram mal funcionamento, decorrente, principalmente, de jumpers de má qualidade.

4.2 CONCLUSÕES

Nesse projeto, foram exercitados conceitos de eletrônico, utilização de timers, USART, PWM, trigonometria, PWM e conversões matemáticas. Em especial, o aprendizado sobre a utilização de sensores ultrassônicos, capturar de borda de subida e servo motores e PWM, expande o portfólio de futuros projetos.

REFERÊNCIAS

1. Artigo/Página de Website (ControllersTech):

CONTROLLERSTECH. HCSR04 Ultrasonic Sensor and STM32. Disponível em: <https://controllerstech.com/hcsr04-ultrasonic-sensor-and-stm32/>. Acesso em: 1 jul. 2026.

2. Artigo de Website Acadêmico/Educacional (Toda Matéria):

TODA MATÉRIA. Lei dos Cossenos. Disponível em: <https://www.todamateria.com.br/lei-dos-cossenos/>. Acesso em: 1 jul. 2026.

3. Vídeo do YouTube:

CRIANDO robôs com sensores ultrassônicos e Arduino. In: Manual do Mundo. YouTube, 18 dez. 2013. Disponível em: <https://www.youtube.com/watch?v=TAAaE7cHdMQ>. Acesso em: 1 jul. 2026.